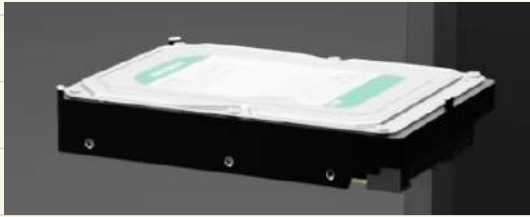


Hard Disk Drives

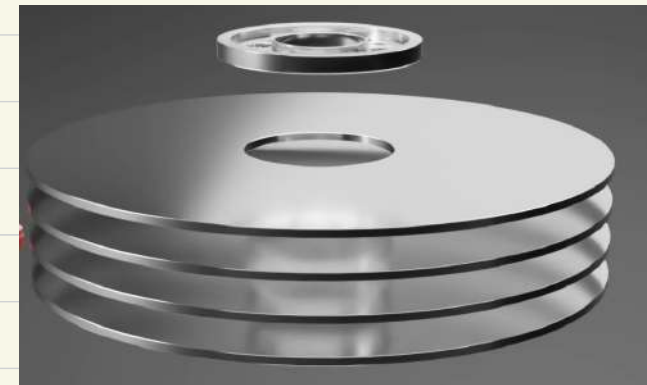


Goal: understand how hard disk drives (HDDs) store data, how do they work mechanically, and how the OS interacts with them?

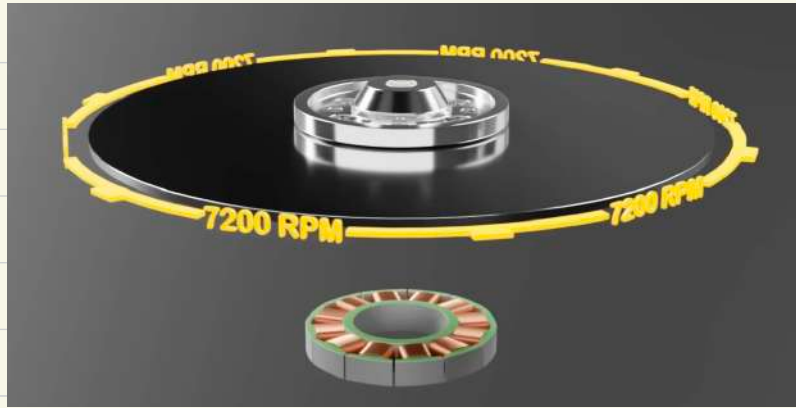
- Hard disk drives are electromechanical devices that store data persistently using magnetic recording.
- they remain crucial for large-scale storage (such as servers, data centers, backups, etc.)

Basic geometry

Platter: a circular hard surface on which data is stored persistently by inducing magnetic changes to it. - a disk may have one or more platters.

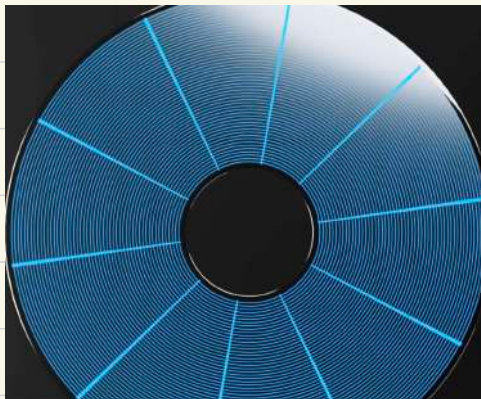
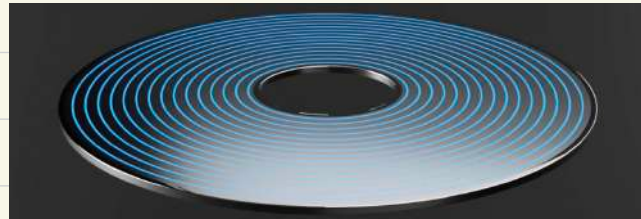


Each platter has 2 sides, each of which is called a surface.
coated with a thin magnetic layer

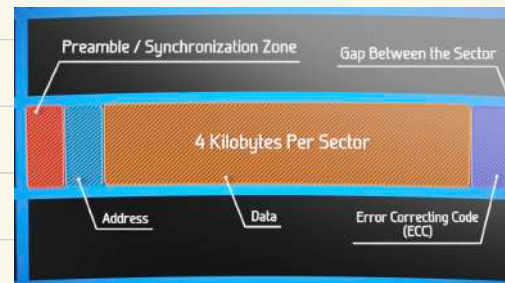


Platters are all bounded together around the spindle, which is connected to a motor that spins the platters around at a constant rate: typically, 7200 - 15,000 rotation per minute (RPM)
10,000 RPM $\Rightarrow$ single rotation takes 6 ms.

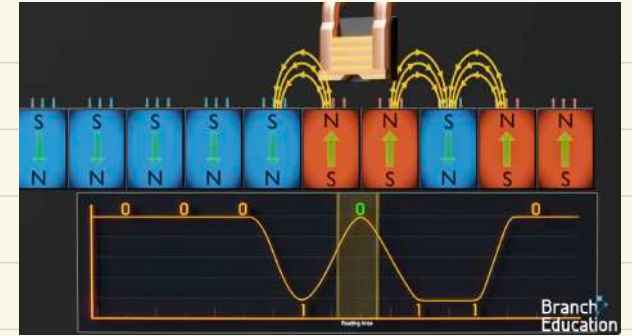
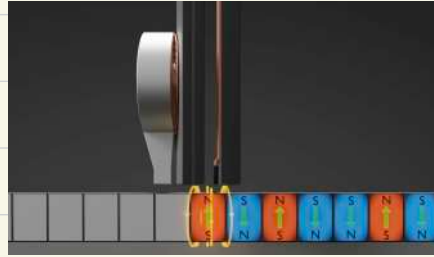
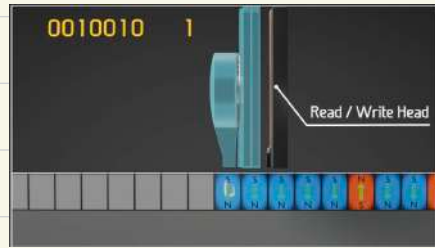
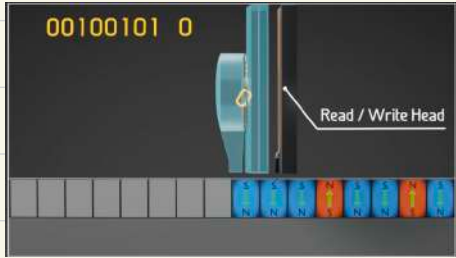
Data is encoded on each surface in concentric circles of sectors. each concentric circle is called a track.



one sector



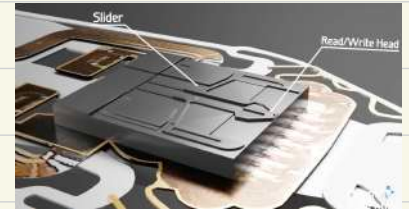
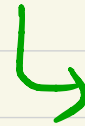
To read, we need a mechanism to sense the magnetic patterns on the disk and
to write, we need to induce a change " " "



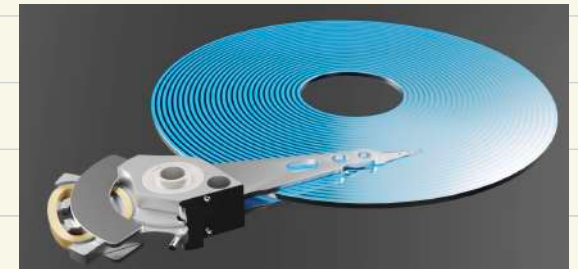
happens while writing

happens while reading.

The disk head accomplishes the process of reading and writing. There is one head per surface of the drive.

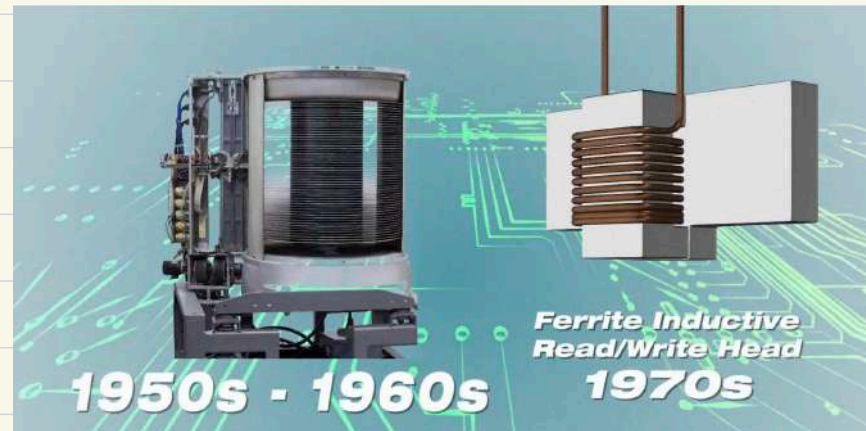


The disk head is attached to a single disk arm which moves across the surface to position the head over the desired track.

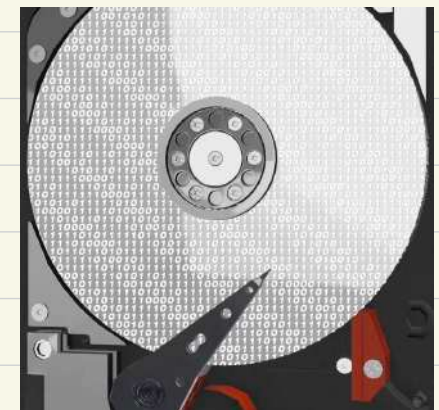


* Picture Credits: Branch Education YouTube channel (for more details on above, refer to their video on How do Harddisk drives work?)

Evolution of Hard disk



First IBM hard disk drive



modern disk
~ 600 Gbits/in²!

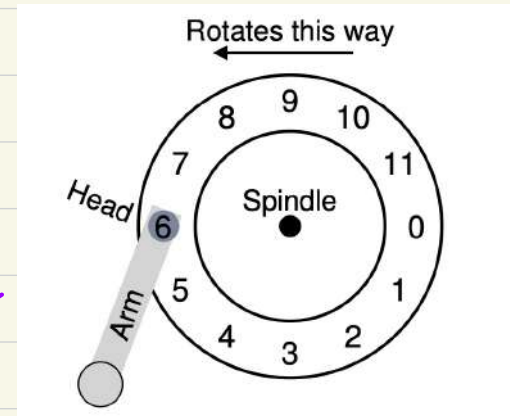
A simple disk drive

Assume a simple disk with a single track.

→ track has 12 sectors, each of which is 512 bytes
in size a typical sector size

→ sectors are addressed by the numbers 0 through 11.

→ disk head is positioned over sector 6



single track latency: The rotational delay.

Suppose we now receive a request to read sector 0.

What would the disk do? → read/write head is already positioned on the correct track (as there is only one track here)

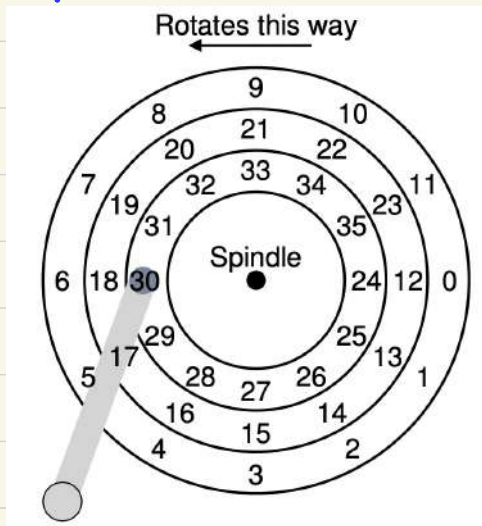
⇒ the only waiting time comes from rotation
the disk must spin until the requested sector comes under the head

This waiting time is called rotational delay or rotational latency.

R → time for a full rotation ⇒ $R/2$ → average rotational delay.

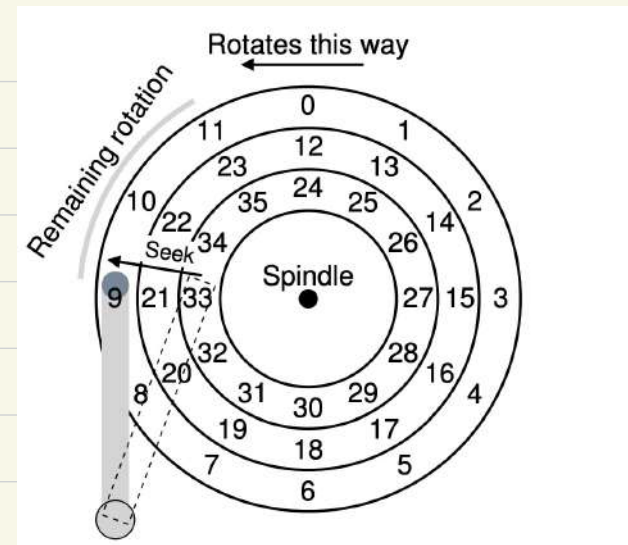
(In the above example, the worst-case request will be for sector 5)

Multiple tracks : seek time



Initial head position → sector 30

a request to
read sector 11



* the drive has to first move the disk arm to the correct track

↳ this process is known as seek

* then do the remaining rotation to reach sector 11.

three phases

Seek

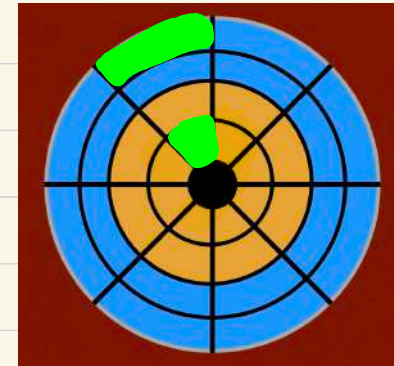
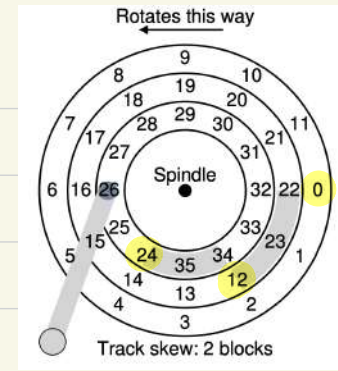
- acceleration → disk arm gets moving.
- coasting → arm is moving at a full speed.
- settling time → carefully position the head on right track.
quite significant, eg!- 0.5 to 2 ms.

Finally, transfer, where data is either read from or written to the surface.
ie, Total I/O time → seek + rotational delay + transfer.

some additional details

track skew: often sectors are skewed because o/w when switching from one track to another, the desired block could have already rotated under the head, and cost an entire rotational delay.

→ make sure the sequential reads are properly serviced even when crossing the boundaries.



multi-zoned disk drives: disk is organized into multiple zones, where each zone is a consecutive set of tracks having same # sectors per track (this division is due to the fact that outer tracks can accommodate more sectors than inner tracks).

cache/track buffer: the small amount of memory (around 8 or 16 MB) which the drive can use to hold data read from or written to disk.

→ allows the drive to quickly respond to subsequent request to the same track.

When does the drive acknowledges the write has completed?

write back caching
(immediate reporting)

write through (after the write
has actually been written to disk)

→ although fast, data could be lost on power failure.

I/O time

Recall that

$$T_{I/O} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}$$

rate of I/O, $R_{I/O} = \frac{\text{size transfer}}{T_{I/O}} \rightarrow \frac{\text{size of the transfer}}{\text{time taken for the transfer}}$

a high performance
SCSI drive

- * spins very fast
- * low seek times

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15,000	7,200
Average Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s
Platters	4	4
Cache	16 MB	16/32 MB
Connects via	SCSI	SATA

a drive built for
capacity

- * cost per byte is
important

For comparison on I/O time, consider two kinds of workload.

- ① random workload $\rightarrow$ issues small (eg: 4KB) reads to random locations on the disk.
- ② sequential workload $\rightarrow$ reads a large # sectors consecutively from the disk without jumping around.

① random 4KB read on cheetah

transfer time with max transfer rate

$$= \frac{4\text{KB}}{125\text{ MB/s}} = \frac{4096\text{ bytes}}{125 \times 10^6\text{ bytes/sec}}$$
$$= 32.768 \times 10^{-6}\text{ s} \sim 30\text{ micro seconds}$$

$$T_{\text{seek}} = 4\text{ms}, \quad T_{\text{rotation}} = 2\text{ms}, \quad T_{\text{transfer}} = 30\text{ microseconds}$$

(15000 RPM $\Rightarrow$ 250 RPS $\Rightarrow$ each rotation takes 4ms
 $\Rightarrow$ on average, disk encounter half rotation $\rightarrow$ takes 2ms)

$$\Rightarrow T_{I/O} = 4\text{ms} + 2\text{ms} + 30\text{ microseconds}$$
$$\sim \underline{\underline{6\text{ms}}}$$

Now, rate of I/O, $r_{I/O} = \frac{\text{size transfer}}{T_{I/O}} = \frac{4\text{KB}}{6\text{ms}}$

$$= \frac{4096\text{ bytes}}{6 \times 10^{-3}\text{ secs}} = 682,666.\bar{6}\text{ bytes/sec}$$
$$= \frac{682,666.\bar{6}}{1,048,576}$$

$$(1\text{MB} = 1,048,576\text{ bytes})$$

$$\approx 0.66\text{ MB/sec} \rightarrow \text{for one 4KB access (seek + rotation every time)}$$

Exercise-verify!

Doing a similar calculation for the Baracuda yields a $T_{I/O}$ of 13.2ms (more than twice slow) and $r_{I/O}$ of about 0.31 MB/s.

Similarly sequential workload (before a very long transfer,
seek + rotation needed only once)

let size of transfer = 100 MB

Exercise: verify

$T_{I/O}$: cheetah = 800 ms, Barracuda = 950 ms

$R_{I/O}$: " = 125 MB/s, " = 105 MB/s

Summarizing, we have,

	Cheetah	Barracuda
$R_{I/O}$ Random	<u>0.66 MB/s</u>	0.31 MB/s
$R_{I/O}$ Sequential	<u>125 MB/s</u>	105 MB/s

* same physical device can perform very differently b/w random and sequential workload.

(almost a factor of 200 difference in Cheetah and a factor of 300 for Barracuda)

* large difference in performance b/w high-end "performance" drives and low-end "capacity" drives.

Disk scheduling

Given a stream of I/O requests, in what order should they be served?

- much different from CPU scheduling
- the position of disk head relative to the request position matters more than length of the job.

The disk scheduler examines the requests and decides which one to schedule next.

Objective: minimize total seek time (and thus total I/O latency)

Idea: Reorder pending requests to reduce unnecessary arm movement.

Example: Imagine a disk with tracks numbered from 0-100 and the head currently at track 100.

let the pending requests arrived be

40, 10, 50, 90, 30 with initial head position at 50.

let us see how different algorithms choose the order to service them.

scheduling strategies

(a) FIFO (First-in First-out)

- serve requests in the order they arrive

Head movement: $50 \rightarrow 40 \rightarrow 10 \rightarrow 50 \rightarrow 90 \rightarrow 30 \approx$ distance covered
= 180 tracks

- simple and fair
- terrible performance (large total head movement)

(b) SSTF (shortest seek time First)

- always service the request closest to the current head position.
(minimize immediate seek distance)

Head movement: $50:50 \rightarrow 40 \rightarrow 30 \rightarrow 10 \rightarrow 90 \approx$ distance covered
= 120 tracks

- good average response time
 - starvation possible (far-away requests might never get served)
- (better than FIFO)

(c) SCAN (Elevator algorithm)

- move the head in one direction (say, outward) servicing all requests along the way, then reverse direction.
(like an elevator going up and down a building)

Head movement : $50 \rightarrow 90 \rightarrow 40 \rightarrow 30 \rightarrow 10 \approx \text{distance covered}$
= 120 tracks.

- avoids unnecessary jumps by sweeping in one direction
- still biased - edge tracks wait longer.

(d) **C-SCAN** (circular scan)

- head moves in one direction (say, outward) servicing all requests until the end of the disk, then returns quickly to the beginning (no servicing on return)
- every requests waits roughly one full rotation

Head movement: $50 \rightarrow 90 \rightarrow \text{jump} \rightarrow 10 \rightarrow 30 \rightarrow 40$

- uniform response time
- slightly higher average seek time than SCAN in some cases.

(e) **LOOK** and **C-LOOK** (optimized variants)

like SCAN, but "looks ahead" - stops at last request (not end of disk)
C-look $\rightarrow$ like C-SCAN, but circular only within request range.

both of these avoid unnecessary travel to unused tracks.

a problem with above scheduling policies → they ignore rotation.

Question: How can we implement an algorithm that approximates STF by taking both seek and rotation into account?

↳ SPTF (shortest positioning Time First)

- chooses the next request that minimizes the total positioning time

includes the seek time → to move the head ^{radially} to the requested track
+ the rotational delay → wait for the requested sector to spin under the head.

Pros: - lower average latency than SSTF (as it accounts for both delays)

- gives better throughput for small, random workloads.

Cons: - more complex: requires rotational phase and seek time model

- starvation risk for requests with long positioning times.

eg:- consider the foll. scenario, where head is at 50 with foll. requests

Request track seek (approx) Rotational wait Total positioning time

A 40 1 ms 3 ms 4 ms

B 30 2 ms 0.2 ms 2.2 ms

C 70 2 ms 2 ms 4 ms

SSTF → picks A (nearest to 50)

SPTF → picks B (soonest completion)

Exercise Other scheduling issues